

# AI INVOICE AUDITOR

Technical guide — React + FastAPI + Google Gemini + MCP + LangGraph. What it does, every file, how to run it, the endpoints, the flow and the rules.

## 1 What the application does

Supplier invoices arrive as PDFs in a folder. For each one the pipeline reads the text out of the PDF, regex reads the fields off that text, and a set of plain rules checks those fields against the ERP — the vendor record and the purchase order — which it fetches from a separate ERP MCP server. **If nothing is wrong, the invoice is stored automatically. If anything is off, it waits for a human**, who approves or rejects it in the UI. When the run is done, a chat opens where you can ask about the invoices; it finds the relevant ones with vector search and Gemini answers from them.

The split is the point: **no model touches the audit**. Reading the fields and deciding the verdict are both plain code that gives the same answer every time and can be explained line by line. Auditing 100 invoices therefore costs nothing and cannot be stopped by a rate limit. Gemini is used for one thing: the chat.

The sample data is 100 invoice PDFs. 92 match their purchase order exactly and are stored automatically; 8 carry one small mistake each and are held for review (section 5).

## 2 Stack

| Layer       | Technology                     | Used for                                                                                                             |
|-------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------|
| UI          | React + Vite                   | frontend/src/main.jsx. One file, hooks only.                                                                         |
| API         | FastAPI + Uvicorn              | Four endpoints on port 8000. CORS allows only http://localhost:5173.                                                 |
| PDF         | pypdf + regex                  | Reads the text out of each invoice PDF and picks the fields off their labels.                                        |
| LLM         | google-genai (Gemini)          | Answers chat questions. The audit never calls it, so a run is free.                                                  |
| ERP         | MCP (FastMCP, streamable HTTP) | A separate server on :8001 with two tools, get_vendor and get_po.                                                    |
| Vectors     | LangGraph + LangChain          | A two-step graph embeds each audited invoice with gemini-embedding-001 into InMemoryVectorStore (which needs numpy). |
| Sample data | reportlab                      | scripts/make_invoices.py draws the 100 PDFs and writes the matching ERP records.                                     |
| State       | Memory only                    | Audited invoices and vectors live in the API process. A restart starts fresh.                                        |

## 3 Every file and its job

| File                              | Job                                                                                                                     |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| backend/app/main.py               | The four endpoints, and REVIEWS — every audited invoice, in memory, keyed by PDF file name.                             |
| backend/app/pipeline.py           | PDF -> text -> regex -> ERP lookup -> validate() -> auto_approved or needs_review. Reading and all the rules live here. |
| backend/app/llm.py                | The chat call to Gemini, and is_up(), the key check behind the Gemini pill.                                             |
| backend/app/erp_client.py         | The MCP client. Each pipeline run opens one MCP session and calls get_vendor / get_po.                                  |
| backend/app/vector_store.py       | The LangGraph ingest (load -> store) and the similarity search.                                                         |
| backend/app/chat.py               | Picks the relevant invoices with vector search and asks Gemini, with a one-line summary of all of them.                 |
| backend/app/data/incoming/        | The 100 invoice PDFs. The only thing the app reads from disk.                                                           |
| backend/mcp_servers/mcp_server.py | The ERP MCP server: loads the ERP files and serves them as two tools.                                                   |
| backend/mcp_servers/data/erp/     | vendors.json (10 vendors) and purchase_orders.json (100 POs). Only the MCP server opens them.                           |
| frontend/src/main.jsx             | The whole UI: the two status pills, run button, review cards, chat, all-invoices table.                                 |
| scripts/make_invoices.py          | Makes the 100 PDFs and the ERP records. Same seed, same files.                                                          |

Logging is print ( ): one line when something happens, prefixed with where it happened — [pipeline], [erp], [vectors], [chat], [gemini], [api]. There is no logging setup module.

## 4 Starting it

Once, from the repository root (PowerShell):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
cd frontend; npm install; cd ..
```

Then three terminals, in this order:

```
# 1. ERP MCP server -> http://127.0.0.1:8001/mcp
python -m backend.mcp_servers.mcp_server
# 2. API -> http://localhost:8000/docs
uvicorn backend.app.main:app --reload --port 8000
# 3. UI -> http://localhost:5173
cd frontend; npm run dev
```

Open the UI at `http://localhost:5173`. On load it asks the API whether the ERP server is up and shows a green or red pill; with that server down, **Run pipeline** is disabled and a banner gives the start command. Press **Run pipeline** and the 100 PDFs are audited — no key is needed for that. Decide the held invoices on the cards. For the chat, paste a Google AI key, press **Check** beside it to confirm it works, and ask.

To regenerate the sample data: `python scripts/make_invoices.py`. It writes the 100 PDFs and the matching ERP records from a fixed seed, so the same files come back every time.

### What the terminals show

ERP server terminal — one line per tool call:

```
ERP MCP server on http://127.0.0.1:8001/mcp - 10 vendors, 100 purchase orders
[erp] get_vendor V-THAM -> found
[erp] get_po PO-3007 -> 2 item(s)
```

API terminal — one line per invoice, then the run summary, decisions and chat:

```
[pipeline] INV-2026-006.pdf auto_approved
[pipeline] INV-2026-007.pdf needs_review PRICE
[vectors] embedded 100 invoice(s), 100 in the store
[api] pipeline run: 100 new invoice(s), 92 stored, 8 need review
[api] INV-2026-007.pdf approved by reviewer
[chat] 'which invoices need review?' -> 8 record(s)
```

## 5 The sample data

`scripts/make_invoices.py` creates 10 logistics vendors (USD, EUR, GBP, INR) and 100 purchase orders, each with 2 to 4 items (air, ocean, road and rail freight, fuel surcharge, customs, handling, storage, insurance). Each invoice bills its own PO, adds VAT/GST for the vendor's currency, and is drawn as a one-page A4 PDF. 92 invoices copy their PO exactly. These 8 carry one small mistake each:

| Invoice      | Mistake              | What the rules report                                                     |
|--------------|----------------------|---------------------------------------------------------------------------|
| INV-2026-007 | Price 7% over the PO | PRICE — HANDLING: billed 122.48 each, the PO price is 114.47              |
| INV-2026-019 | One unit too many    | QUANTITY — FRT-ROAD: billed 3, the PO ordered 2                           |
| INV-2026-028 | Total off by 25      | TOTAL — lines plus tax come to 4,991.06 but the invoice total is 5,016.06 |
| INV-2026-041 | Item not on the PO   | ITEM — DOCS is billed but not on PO-3041                                  |
| INV-2026-056 | Price 7% over the PO | PRICE — INSURANCE: billed 95.02 each, the PO price is 88.80               |
| INV-2026-063 | One unit too many    | QUANTITY — FRT-RAIL: billed 2, the PO ordered 1                           |
| INV-2026-077 | Total off by 25      | TOTAL — lines plus tax come to 1,579.02 but the invoice total is 1,604.02 |
| INV-2026-092 | Item not on the PO   | ITEM — DOCS is billed but not on PO-3092                                  |

Every mistake is small and produces exactly one finding, which is what a reviewer would see in real accounts payable: a price that crept up, an extra unit, a fee nobody ordered, a total that does not add up. The list is the `MISTAKES` dict at the top of the script.

## 6 The flow

### A — Run pipeline

- 1. The UI POSTs /pipeline/run. The API first checks the ERP server is up; if not, it answers 503 with the start command.
- 2. pipeline.run() lists the PDFs in backend/app/data/incoming it has not processed yet.
- 3. Eight at a time, in a thread pool: pypdf reads the text and extract() picks the fields off their labels with regex. No network call, so this step cannot fail on a key or a rate limit.
- 4. erp\_client.lookup() opens **one** MCP session and fetches every vendor and PO the batch needs — 10 get\_vendor and 100 get\_po calls for the sample data.
- 5. validate(invoice, vendor, po) runs every rule. No findings: status **auto\_approved**, the invoice is stored. Any finding: **needs\_review**.
- 6. The API keeps each review in REVIEWS, and vector\_store.index() embeds them through the LangGraph graph — the one part of a run that needs the key, and it is skipped without one.
- 7. The UI shows the counts, a card for each held invoice with what is wrong, and opens the chat with a summary of the run.

### B — A human decides

- 1. The reviewer reads the finding on the card and clicks Approve or Reject.
- 2. The UI POSTs /reviews/{file}/decision with approved or rejected.
- 3. The API refuses anything not in REVIEWS (404), not waiting for review (409), or any other decision (422). Otherwise the status becomes approved or rejected and the invoice is re-embedded so the chat sees the new status.
- 4. The card leaves the queue and the counters update.

### C — Chat

- 1. The UI POSTs /chat with the question and the key. Without a key the chat says it needs one.
- 2. vector\_store.search() returns the 8 invoices closest in meaning to the question.
- 3. chat.py sends Gemini the question, those 8 records, and a one-line summary of all audited invoices: how many per status, and which ones need review, were approved or were rejected.
- 4. The answer comes back with the invoice numbers it was built from, shown under it as sources.

## 7 The rules

All in validate(invoice, vendor, po) in pipeline.py. It is a pure function — no network, no model — so it can be tested with plain dicts and always gives the same answer.

| Code          | Flagged when                                                                                |
|---------------|---------------------------------------------------------------------------------------------|
| MISSING_FIELD | invoice_no, invoice_date, vendor_id, po_number, currency or total_amount could not be read. |
| TOTAL         | The line amounts plus tax differ from the invoice total by more than 0.01.                  |
| VENDOR        | The vendor is not in the ERP, or is marked inactive.                                        |
| PO            | The purchase order is not in the ERP. The item checks below are skipped.                    |
| PO_VENDOR     | The purchase order belongs to a different vendor.                                           |
| CURRENCY      | The invoice currency differs from the PO currency.                                          |
| ITEM          | A billed item code is not on the purchase order.                                            |
| QUANTITY      | A billed quantity differs from the quantity ordered.                                        |
| PRICE         | A unit price is more than 5% away from the PO price (PRICE_TOLERANCE).                      |

**The decision is one line:** no findings means auto\_approved, any finding means needs\_review. There are no scores and no severities; a human looks at every invoice that is off, whatever is wrong with it.

## 8 API endpoints

| Endpoint                      | Body                                   | Returns                      | Purpose                                                                                                            |
|-------------------------------|----------------------------------------|------------------------------|--------------------------------------------------------------------------------------------------------------------|
| POST /health                  | api_key (optional)                     | erp_mcp, gemini (true/false) | The two outside services. The UI asks for the ERP on load; the key is checked only when you press Check beside it. |
| POST /pipeline/run            | api_key (optional)                     | reviews[]                    | Audit every new PDF, embed the results. 503 when the ERP server is down.                                           |
| POST /reviews/{file}/decision | decision: approved   rejected, api_key | review                       | A human decides a held invoice. 404, 409, 422 as in section 6.                                                     |
| POST /chat                    | question, api_key                      | answer, sources[]            | Answer a question about the audited invoices.                                                                      |

A review is {file, invoice, findings[], status}. An invoice is invoice\_no, invoice\_date, vendor\_id, vendor\_name, po\_number, currency, tax\_amount, total\_amount and line\_items (item\_code, description, qty, unit\_price, total). A finding is {code, message}. Swagger UI at /docs has a worked example for every request body.

The API key falls back to the GEMINI\_API\_KEY environment variable, so Swagger and curl work without sending it. /health is a POST rather than the usual GET for one reason: the key belongs in the body. In a query string it would be written to the server log on every check.

## 9 The ERP MCP server

| Tool                  | Returns                                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------------------------------|
| get_vendor(vendor_id) | The vendor record — vendor_id, name, currency, active — or {} if unknown.                                         |
| get_po(po_number)     | The purchase order — po_number, vendor_id, currency and items with item_code, qty, unit_price — or {} if unknown. |

**Why a separate server.** These are the only calls that leave the auditor and reach the system of record. Behind MCP, the ERP side could be owned and secured by another team and reused by other agents, and swapping the mock for a real ERP changes the server, not the rules. Nothing in backend/app opens the ERP files, so there is no way around the boundary.

**Why streamable HTTP.** It is an ordinary service: its own terminal, its own port, its own log of tool calls, restartable on its own. A stdio server would be a child process of the API instead.

**Why one MCP session per run.** Opening an MCP session costs about 150 ms (the HTTP client setup); a call on an open session costs about 8 ms. One session per lookup made a 100-invoice run take 13.7 s; reading all the PDFs first and fetching every vendor and PO over one session brought it to 0.9 s. It also moved the network out of validate(), which is why that function is pure.

**When it is down.** The UI shows a red pill and the start command, and Run pipeline is disabled. POST /pipeline/run answers 503. Nothing falls back to reading ERP files locally.

## 10 Vectors and the chat

```
ingest = StateGraph(IngestState).add_sequence([load, store]).add_edge(START, "load").compile()
```

| Step  | What it does                                                                                                                                                                 |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| load  | Each review becomes one LangChain Document: a few sentences with the invoice, its items, its status and its problems. The id is the PDF file name.                           |
| store | add_documents() embeds the documents with gemini-embedding-001 and saves them under their ids. Saving an id again overwrites it, which is how a decision updates the record. |

**One invoice, one vector, no chunking.** A record is a few hundred characters that this system wrote itself. Splitting it would put a problem in one chunk and the invoice number in another, and a top-k search would return fragments instead of invoices.

**Embedding types.** LangChain's Gemini embeddings embed stored documents as RETRIEVAL\_DOCUMENT and questions as RETRIEVAL\_QUERY by default, so one embeddings object is enough.

**The summary line.** Vector search sends Gemini 8 records, so on its own it could not answer “how many invoices need review?” across 100. chat.py adds one line with the counts per status and the invoice numbers that need review, were approved or were rejected. Retrieval handles “which” and “why”; the summary handles “how many”.

**Where the vectors live.** One InMemoryVectorStore in the API process, created on the first run with a key and gone when the API stops. A run without a key embeds nothing, so the chat then answers from the summary line alone. Moving to FAISS, Chroma or pgvector changes the one line that creates the store.

## 11 Reading the invoice, and why no LLM does it

**extract()** is the whole reader. pypdf turns the page into text, one value per line, and two patterns pick the fields off it: after ( label ) takes the value printed after a label (Invoice No:, Vendor ID:, PO Number:), and one line-item pattern matches the five cells of a table row — code, description, qty, unit price, amount. number ( ) turns “1,200.00” into 1200.0 and anything unreadable into None, so a field that is missing stays missing and MISSING\_FIELD reports it instead of a guess.

**An earlier version sent every PDF to Gemini with a JSON schema.** It worked, and it was removed: 100 model calls per run, a key needed before anything could start, a rate limit able to stop a run halfway, and a reply that had to be checked field by field. Regex gives the same 92 / 8 verdict on the same PDFs, in 0.74 seconds, for nothing.

**What that trade costs.** This reader knows one layout, because these PDFs come from scripts/make\_invoices.py. Real invoices arrive in a hundred layouts, and that is exactly where an LLM earns its call — or a layout-aware extractor such as Document AI. The honest statement is that the reader is swappable: it is one function, extract(path) -> dict, and nothing downstream knows which one ran.

## 12 The UI

| Part         | What it shows                                                                                                                              |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Header       | Title, two status pills — the ERP server, and the Google AI key once checked — and Run pipeline, disabled while the ERP server is down.    |
| Banner       | When the ERP server is down: the start command and a Check again button. Also shows API errors.                                            |
| Key bar      | The Google AI key, kept in this tab's sessionStorage only, and a Check button that tests it. Nothing is sent to Gemini until you press it. |
| Stats        | Invoices, stored automatically, need review, approved by you, rejected.                                                                    |
| Review cards | One per held invoice: number, vendor, PO, total, each finding with its code, Approve and Reject.                                           |
| Chat         | Opens after the run with a summary of it. Answers show their source invoice numbers.                                                       |
| All invoices | A collapsible table of all 100 with their status.                                                                                          |

## 13 What is kept, and where

| What                              | Where                                  | After a restart                         |
|-----------------------------------|----------------------------------------|-----------------------------------------|
| Audited invoices and their status | REVIEWS in main.py                     | Gone; the next run audits all 100 again |
| Which PDFs were processed         | PROCESSED in pipeline.py               | Gone                                    |
| Vectors                           | InMemoryVectorStore in vector_store.py | Gone; rebuilt on the next run           |
| Invoice PDFs                      | backend/app/data/incoming              | Kept — input, never written by the app  |
| ERP records                       | backend/mcp_servers/data/erp           | Kept — read only by the MCP server      |

“Stored” means kept in the API’s memory until it stops, so every run starts from the same place. Moving REVIEWS into a database is what makes a decision survive a restart.

## 14 Security

- **API key.** In the browser's sessionStorage, so it dies with the tab. Sent to the local API in the request body, never in a URL, never logged or written to disk.
- **Model output.** No model output reaches the rules: the fields come from regex, so a verdict can never be a model's invention.
- **Chat grounding.** Gemini sees only the retrieved records and the summary line — no files, no ERP, no web.
- **ERP.** Two read-only tools. The app has no code path to the ERP files.
- **Requests.** Decisions are limited to approved or rejected, and only for invoices waiting for review. CORS allows only the Vite origin.
- **Access.** No login: anyone who can reach the API can approve an invoice, and a decision does not record who made it.